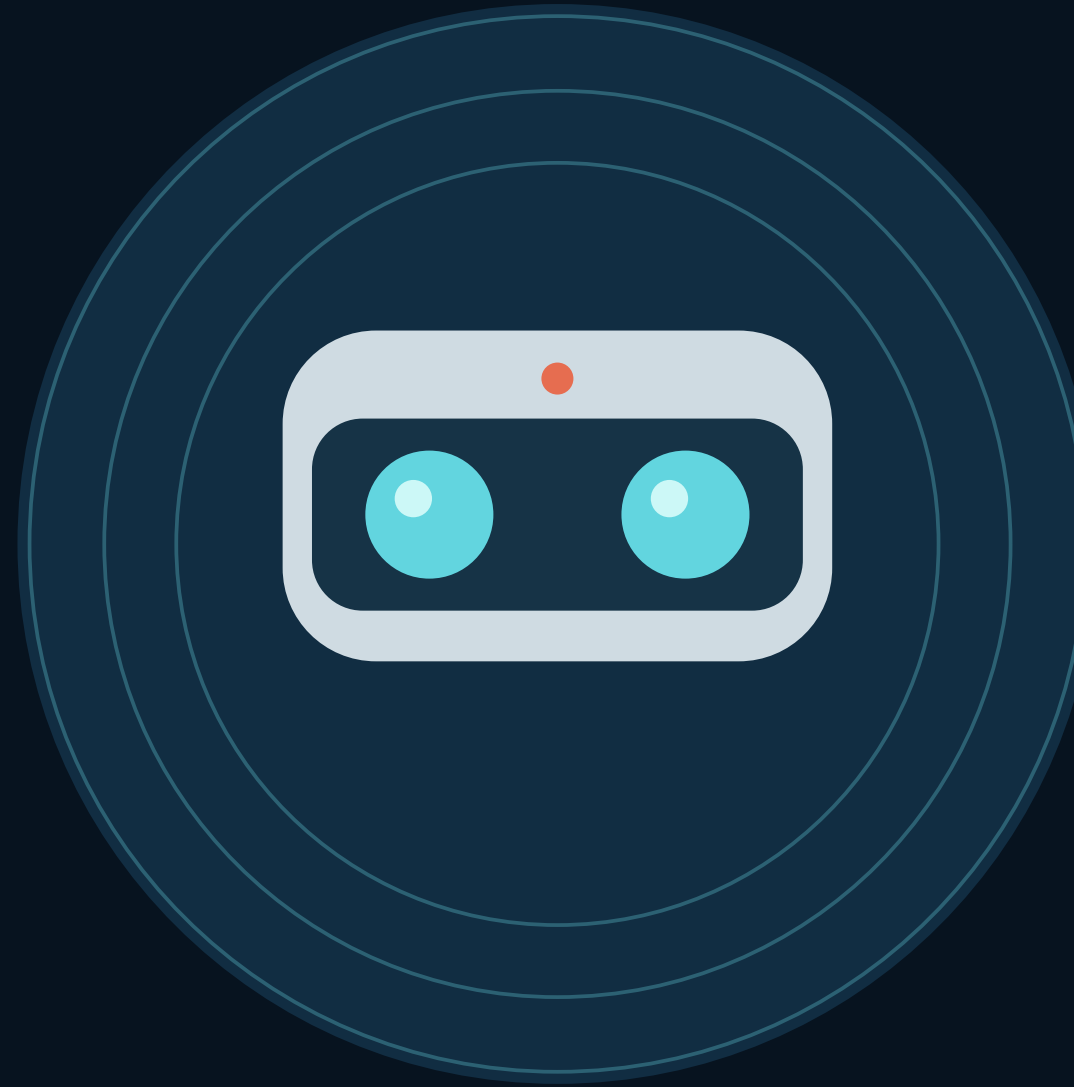


R.O.B. VISION

OPTICAL ROBOT COMPANION / MK.01



TECHNICAL REFERENCE

Architecture, optics, networking, hardware, and verification

CURRENT REFERENCE / 24 SEPTEMBER 2026

UNO Q + RETROPIE LIVE · CAMERA VALIDATION PENDING

CHAPTER 01 / CURRENT REFERENCE

R.O.B. Vision technical architecture

Status, 24 September 2026: The Arduino UNO Q runs the App Lab controller, matrix sketch, HTTP dashboard, and virtual R.O.B. model. RetroPie launch hooks and a paired virtual Controller 2 receiver run on `retropie.local`. Both Gyromite gate colors responded in Game A. Optical decoding has synthetic timing evidence, but no attached UNO Q camera/display has been validated. R.O.B. and all accessories exist only in software.

Runtime path

```
RetroPie runcommand -- authenticated launch/exit --> UNO Q controller
Game display -- light flashes --> UNO Q camera --> optical decoder
                                --> virtual model
                                +-- HTTP snapshot --> browser
                                +-- Gyromite pads --> RetroPie
                                    virtual Controller 2
```

The App Lab `python/main.py` publishes port 80 and forwards to `controller/service.py` on port 8766. Its Router Bridge sends controller status and action hints to `sketch/sketch.ino` for the UNO Q LED matrix. The dashboard polls `/api/state` every 500 ms. The local file preview uses its separate JavaScript demonstration model; a served page with an available controller enters live mode.

Authority and state

`controller/service.py` owns the selected game, camera/Test status, recent events, and virtual game models. `/api/state` is a snapshot with schema version 1, game, robot, camera, recent events, link, and Test data. Recent events are capped at 30; they are display history, not a durable event stream. There is no session ID, command queue, WebSocket, or emulator game-state hook. Reopening the browser reads a fresh snapshot without resetting the controller.

`controller/model.py` keeps Gyromite's two gyros and Stack-Up's five blocks. A complete optical command or operator command passes through the same model. Invalid moves return an error and leave piece state intact. Stack-Up starts with all five blocks on Tray 3; a grip can carry a block with those above it, preserving order. The current live model uses this initial layout for all modes; mode-specific Bingo layouts are not implemented. Gyromite's 55-second spin lifetime and 60-second Fast Gate hold are simulation settings, not measured Nintendo hardware properties.

The Gyromite pad reducer derives red and blue from gyro location/spin or a held unspun gyro. The RetroPie service polls the UNO Q, creates a Linux uinput controller, and applies the current states only while Gyromite is the active RetroArch game. It releases both buttons on exit, unknown game, missing process, failed authentication, or stale

state. The verified RetroArch port 2 mapping makes the dashboard's blue control move a blue gate and red control move a red gate.

Optical capture

`controller/optical.py` recognizes the ROM-derived six movement/grip patterns from timestamped bright/dark samples. `controller/service.py` optionally opens an OpenCV capture device, requests 60 fps, samples the central 60% by default, and reports measured fps. Test-mode alternation drives the red status light without moving R.O.B. A complete valid command drives one action; ambiguous or partial traces drive none. Synthetic 60 fps jitter runs yielded 667 correct and 233 rejected commands out of 900, with zero wrong commands in that run. At 30 fps, all 900 were rejected. This is a decoder result, not a live camera acceptance result. An emulator hook is an unimplemented fallback if camera timing proves inadequate.

HTTP and trust boundary

Browser controls on the trusted LAN can call `/api/game`, `/api/command`, `/api/gate-assist`, `/api/test/arm`, and camera actions without a token. The browser also reads `/api/state`, `/api/camera/frame`, and matrix status. The service uses JSON and same-origin checks for browser writes. `/api/launch` and receiver identity require the shared token; pairing uses a time-limited code and verified TLS certificate fingerprint to deliver it. Do not expose port 80 or 8766 to an untrusted network.

Setup's **Reset & Reconnect** closes and reopens OpenCV capture; it does not reset the USB hub. UNO Q host recovery units are supplied under `deploy/uno-q` but are not installed by App Lab and currently need separate privileged setup. The camera is not attached in the tested installation.

Resets and limits

Selecting a game or **Home** initializes that game's virtual pieces. **Emergency Stop** in live mode stops camera capture and clears the selected game; in the file preview it cancels its local animation. Game exit clears context and releases buttons. The RetroPie receiver also scans the running RetroArch process and resends game identity after a UNO Q restart. It does not infer Hector's position, gate animation, Stack-Up score, or game victory.

See [configuration](#), [network architecture](#), [optical input](#), and the [verification report](#).

CHAPTER 02 / CURRENT REFERENCE

Original R.O.B. manual: historical notes

Source: [R.O.B. instruction manual transcript](#) at Atari HQ. This is a paraphrased reading of the original physical toy, not a hardware specification for R.O.B. Vision.

The original robot grasped/released, raised/lowered, and rotated/carried objects. Its operator aimed its eyes at a CRT test signal. A head light indicated optical reception and readiness, went out while it moved, and returned when ready for another command. R.O.B. Vision represents those movements and states in a browser; its UNO Q camera is separately mounted and watches a modern LCD/OLED. There is no physical R.O.B. head, arm, or accessory.

The manual warns about glare, fluorescent light, blocked sight lines, overlays, and overly bright images. Those remain useful camera-test cases. The modern UI must separate **flash visible** from **complete command decoded**, and show TEST → READY → BUSY → READY clearly. The original approximate CRT distance is only historical context; actual camera framing and exposure need calibration on the chosen display.

The original manual's battery, mechanical, and spinning-gyro safety instructions apply to Nintendo's toy. This virtual project instead needs robust command validation, correct virtual piece state, a fresh paired Gyromite return link, and clear simulation/live labels. See [optical input](#) and the [virtual system contract](#).

CHAPTER 03 / CURRENT REFERENCE

Gyromite manual: design and test notes

Source: [Gyromite instruction booklet transcript](#) at Digital Press. This is a paraphrased summary of Nintendo's physical game accessory, translated into rules for a virtual R.O.B. The transcript includes editorial notes and possible transcription mistakes; confirm game behavior with a user-supplied game before claiming compatibility.

Original accessory and virtual equivalent

The original set has one motorized spinner, two gyro holders, red/blue gyro trays, removable hands, and a holder for Controller 2. A gyro on the blue tray lowers the blue gate; one on red lowers the red gate. The original trays press Controller 2 mechanically. R.O.B. Vision draws these objects and derives a **virtual** red/blue button state in UNO Q software, then sends it over Wi-Fi or Ethernet to a paired virtual Controller 2 receiver. There are no physical gyros, trays, or contact sensors in this project. The transcript does not unambiguously map tray colors to Controller 2 A/B. On the RetroPie test setup, an isolated blue virtual pad press moved the blue Game A gate after its A/B mapping was corrected; the player later confirmed that both red and blue controls work.

The booklet says that **one gate can be operated without spinning a gyro**. R.O.B. may hold an unspun gyro down on a pad, then lift it to release. When both gates need control, a spinning gyro remains upright on one pad while R.O.B.'s hands move the other. The original setup begins with both gyros in holders, and the instructions describe R.O.B. moving a gyro between holder, spinner, and tray. The virtual controller therefore treats holders as resting positions and the spinner as an optional step driven by game needs.

Game modes

Mode	Manual behavior	Virtual design implication
Test	Optical aiming signal	Provide camera framing and decoder diagnostics before play.
Direct	Up/down/left/right/open/close go straight to R.O.B.	Validate six optical commands and show each virtual movement.
Game A	Player controls the professor on a dark screen; Start enters a blue robot-transmission screen	Decode complete robot commands in the correct mode; avoid claiming to know professor position from flashes.
Game B	Professor walks while Controller 1 commands R.O.B. directly	Show ready/busy state and virtual-pad-to-game latency.

Acceptance questions

- Capture Test and six Direct commands on the intended LCD/OLED and emulator.
- Demonstrate held unspun press/release of one virtual pad.
- Demonstrate two independent gyro positions and spin-down phases, red/blue button state, and return to holders.
- Measure red/blue-to-A/B mapping, receiver freshness, and actual game gate response. A local visual button state is not proof that the emulator accepted input.

- Test Game A's transmission switch and Game B's direct command cadence.

Original booklet warnings about touching real spinning gyros are historical context. The [virtual system contract](#) is the current product scope.

CHAPTER 04 / CURRENT REFERENCE

Stack-Up manual: design and test notes

Source: [Stack-Up instruction booklet scan](#) at Digital Press. This paraphrases the original accessory and game rules. R.O.B. Vision models the robot and pieces virtually; no physical tray or block is built.

Virtual set and starting layouts

The booklet shows five colored ring-shaped pieces—red, white, blue, yellow, and green—on five numbered trays around R.O.B. Tray 3 is directly in front. Direct and Memory begin with all five on Tray 3, top to bottom red, white, blue, yellow, green. One-player Bingo also begins with five on Tray 3, in any color order. Two-player Bingo begins with three on Tray 3 and one each on Trays 2 and 4. The current UNO Q model initializes the Direct/Memory Tray 3 stack for every Stack-Up session. Bingo-specific starting layouts are historical requirements, not implemented profiles. At the start of a virtual session, the pose is centered over Tray 3, at the highest level, with hands open.

Command and stack behavior

The game sends one of six movement commands—left, right, up, down, open, close—per completed signal. The original robot has five tray stations and six height levels; Stack-Up up/down moves one level, unlike Gyromite's two-level movement. Closing at a lower block can lift that block and every block stacked above it together. The virtual controller therefore carries an ordered **stack segment**, not just one block. Opening places that segment, preserving color order; all five blocks remain accounted for. The game does not send a block ID, destination tray, target pattern, or confirmation that the player's arrangement is correct. Those are inferred from our virtual pose and stack state or seen by the player on the game screen.

The current scripted preview applies one virtual command per step. It first moves red alone from Tray 3 to Tray 4, then closes around blue at level 3 and carries blue plus white to Tray 2. The blocks keep their bottom-to-top order, and all five remain accounted for. Local command buttons use the same virtual stack model. This is not evidence that the camera decoded any command; live play must animate each received primitive independently, including repeated commands and Memory playback.

Modes

Mode	Booklet behavior	Virtual design implication
Test	Optical aiming signal	Reuse camera calibration and readiness check.
Direct	Professor lands on six command keys	Decode and display left, right, up, down, open, or close. Player scores/advances with Start after arrangement.
Memory	Up to 100 programmed commands at selected speed, then END	Measure cadence; the current model applies commands immediately and has no bounded queue. Report misses during camera validation. END is a programming marker, not a robot movement flash.
Bingo, one player	Row or column completion sends a command	Apply only a complete validated optical command.
Bingo, two players	Two human controllers and a different starting layout	Validate separately; keep human controller input on the cabinet.

The booklet says a simultaneous completed row and column is a no-command condition until one line changes. Do not turn that pattern into two virtual actions. The flash stream does not report a target pattern or a victory result. The UNO Q knows its own modeled block stacks but cannot infer the game's scoring outcome without another source. In Direct, Memory, and one-player Bingo, the human presses Start to request scoring after matching the displayed arrangement; a successful animation alone does not advance a round.

Stack-Up's documented modes have no Gyromite-style accessory that presses Controller 2. Its tray model needs no button return path to the game. The browser preview now demonstrates both a one-block transfer and an ordered two-block carry. The live virtual model conserves five blocks and updates a placement on a valid open command. Optical Memory timing still needs hardware validation.

See the [virtual system contract](#), [ROM signal analysis](#), and [verification plan](#).

CHAPTER 05 / CURRENT REFERENCE

Game identification from RetroPie launch events

Status: installed on the dedicated `retropie.local` test machine. A real Gyromite launch selected the game on the UNO Q, and a subsequent real exit cleared it. Controlled start/end calls selected and cleared both games. The game still flashes commands for the camera to decode. Launch identification supplies context and does not replace optical input.

Decision

Use the same approach as VirtualGlove: a small RetroPie runcommand launch/end hook sends the game identity. On the dedicated test machine there were no existing custom hooks, so `launch` and `end` scripts were installed. At launch, RetroPie supplies system, emulator, ROM path, and command. Match the **exact ROM basename including .nes, .zip, or .7z**, case-insensitively, against an explicit registry. At exit, clear the active game. On another machine, merge with any existing hook rather than overwriting it.

Only `nes` and `famicom` are eligible. An unregistered filename or other system resolves to idle, not a guessed game. A changed archive filename needs an explicit registry entry. This avoids recognizing an unrelated game merely because its name contains “Gyromite” or “Stack-Up.”

The checked-in registry covers the user's Gyromite (World) and Stack-Up (World) .7z and .zip files plus their named .nes members. A read-only inspection of the supplied archives on 24 September 2026 found one 40,976-byte .nes member per archive. For each game, its .zip and .7z members had identical bytes. No ROM bytes, extracted files, or archive copies were added to R.O.B. Vision.

Resolver and live check

The `resolver` accepts the same four launch arguments used by RetroPie and prints a small JSON event. The `sender` authenticates to the UNO Q using the token file and sends the event with bounded retries. On 24 September 2026, a real EmulationStation Gyromite launch used the .7z archive and selected `gyromite` on the UNO Q. The next real exit cleared the selected game. Controlled calls of the installed hooks also selected and cleared both games.

```
python3 tools/identify_game.py start nes lr-fceumm "/roms/nes/Gyromite (World).zip" ""
python3 tools/identify_game.py start nes lr-fceumm "/roms/nes/Stack-Up (World).7z" ""
python3 tools/identify_game.py end
```


The first two calls identify `gyromite` and `stack_up`; the last returns `idle`. The ROM argument is the **launched filename** reported by RetroPie, not the inner `.nes` member of an archive unless RetroPie actually launches that extracted file.

Current cabinet-to-robot path

```
RetroPie runcommand start/end
-> exact local registry match
-> authenticated LAN game-state event
-> UNO Q selects and displays game/accessory context
-> optical test + virtual accessory profile checks
-> READY for validated screen-flash commands
```

The sender transmits system, ROM path, and launch/exit event over authenticated HTTP with bounded retries. The RetroPie receiver also scans the active RetroArch process and resends identity if the UNO Q restarted during a running game. A launch event is game context only; it does not claim camera calibration or a decoded movement. Browser manual selection is also available while no supported game is active. There is no implemented comparison between camera-observed flashes and a launch identity, and no emulator-side command hook.

Gyromite's paired tray-to-Controller-2 return path remains separate from this launch notification. Stack-Up needs no gyro-style button return in its documented modes. Both games may use the Wi-Fi/Ethernet launch event for context while receiving their movement commands through the camera.

Remaining validation

1. Simulate all six registered archive/member basenames, an unrelated NES ROM, and another system; verify only intended games match.
2. Confirm the remaining archive formats and Stack-Up through the real RetroPie UI; the Gyromite `.7z` launch/exit and controlled hook calls have already been checked.
3. Verify start, exit, unknown game, back-to-back game launches, missed acknowledgement, and sender/robot restart. No stale game may stay armed.
4. Confirm all existing cabinet runcommand actions still execute in their original order and that hook failure never prevents the game from starting.
5. Validate camera-observed Test and movement flashes for both selected games on the intended display.

See [network architecture](#), [optical input](#), [configuration](#), and [verification plan](#).

CHAPTER 06 / CURRENT REFERENCE

Optical game input: camera watching a modern display

Current implementation

The UNO Q's optional OpenCV capture path watches the cabinet's LCD/OLED game display. It requests 60 fps, timestamps delivered frames, samples normalized luminance from the central 60% rectangle by default, and passes samples to `controller/optical.py`. Setup shows a low-rate camera preview with the sampling rectangle, measured fps, and signal level. A CLI `--camera-roi x,y,width,height` can change the crop. The browser does not currently offer a draggable crop, raw brightness trace, or automatic display locator.

The decoder uses the ROM-derived `000101w1x1y1z` command family and a 60 Hz source-cell model. Six commands map to left, right, up, down, open, and close; Gyromite moves two levels vertically while Stack-Up moves one. Test-mode alternation blinks R.O.B.'s red status light, and the ready-light command glows steadily without moving pieces. Only complete, unambiguous transmissions cause one model action. Partial or ambiguous samples are rejected. Separate repeated transmissions remain valid commands.

Measured boundary

Synthetic stress tests with timing jitter at 60 fps decoded 667 of 900 valid command traces and rejected 233; none in that run became a wrong command. At 30 fps, all 900 were rejected. Ten further 900-trace seeds also produced zero wrong commands. These results show that conservative rejection works, but they do **not** establish reliable play. No actual camera is attached to the tested UNO Q, and no screen, exposure, frame timestamps, or live game Test signal has been validated. A nominal 60 fps setting does not guarantee 60 delivered frames or alignment with one-frame flashes. Frame drops, rolling shutter, display persistence, and video filters can change the signal.

Commissioning test

1. Attach the intended camera, point it at the game's flash region, and use Setup's **Start Camera Check**. Check the delivered fps and green sample rectangle.
2. In each game's Test mode, choose **Watch Test Flashes**. Confirm sustained alternating flashes blink the virtual red light without causing movement. Repeat under normal room lighting and beside the UNO Q matrix.
3. Capture each of the six commands many times in both games, plus ready signals and idle gameplay. Compare accepted/rejected actions with what the game actually displayed. Include Stack-Up Memory speed and Bingo's no-command simultaneous row/column case.

4. Change display scaling, exposure, glare, and camera aim slightly. Record measured fps, dropped frames, decode counts, and false triggers. A proposed project target is at least 99 correct in 100 transmissions per command with zero idle false triggers; it has **not** been met.
5. If this camera path cannot meet the target at the available 30–60 fps, consider an authenticated emulator command hook. That hook is not implemented.

Nintendo's [photosensing patent](#) and the [historical manual](#) explain the original CRT light mechanism. They do not prove a modern camera/display combination will work. The [ROM analysis](#) gives the exact patterns; the [verification report](#) separates synthetic results from hardware results.

CHAPTER 07 / CURRENT REFERENCE

Gyromite and Stack-Up ROM signal analysis

Status: Static analysis of the user-supplied Gyromite (World) and Stack-Up (World) ROMs, 24 September 2026. The ZIP and 7z copies of each game contain identical NES data. No ROM or extracted graphics is stored in this project. The derived patterns have not yet been captured from the user's LCD/OLED and camera.

Run `python3 tools/analyze_rob_roms.py --gyromite '/path/to/Gyromite (World).zip' --stack-up '/path/to/Stack-Up (World).zip'` to reproduce the command report from the archives in place. The program rejects unexpected ROM layout or changed evidence bytes. The inspected full-file SHA-256 hashes are `bf1b323ba39c84b964f93127598b267ff3f16cbf99c533ae1e9b8332232b3e5b` (Gyromite) and `3ab6bc99246783a6bf7083481027fe358ccdb147dcf1165eaf08aaf4e1b06548` (Stack-Up). Each is an iNES mapper-0 image with 32 KiB program ROM and 8 KiB character ROM.

What the games transmit

The screen flashes represent **one robot primitive per command**, not a complete task such as “move the red gyro to its tray” or “solve this Stack-Up pattern.” A 13-frame message has the form:

`000101w1x1y1z`

0 is the dark frame (NES palette \$0F), 1 is the green frame (\$2A), and wxyz selects an action. The pattern is consistent with the independently documented [NESdev R.O.B. interface](#) and [original hardware protocol research](#). At a roughly 60 Hz NES video rate, the 13 data frames span about 217 ms; the final visible duration and gaps still depend on emulator and display behavior. Stack-Up's transmission routine schedules an additional dark frame after the 13 data frames.

Action	wxyz	Full 13-frame pattern	Gyromite	Stack-Up
Open grippers	1010	0001011101110	Yes	Yes
Close grippers	0110	0001010111110	Yes	Yes
Turn left one station	0100	0001010111010	Yes	Yes
Turn right one station	1000	0001011101010	Yes	Yes
Raise two levels	0101	0001010111011	Yes	No
Lower two levels	1101	0001011111011	Yes	No
Raise one level	1100	0001011111010	No gameplay table	Yes
Lower one level	0010	0001010101110	No gameplay table	Yes
Head ready light on	1001	0001011101011	ROM command table	Stack-Up test path

The eight-bit command byte is the message after the five leading bits: for example, open is \$EE, close \$BE, left \$BA, right \$EA, Gyromite up/down \$BB/\$FB, Stack-Up up/down \$FA/\$AE, and ready light \$EB. The ready-light message is a status operation. It must never move an axis.

Test mode also uses sustained alternating dark/green flashes, distinct from the 13-frame ready-light command. The virtual head light blinks only after detecting a sustained alternation from the camera; the ready-light command produces a steady light. These are status indications, not motion commands. The Test-mode flash detector must be checked against real camera traces to confirm timing on the intended LCD/OLED and emulator.

ROM evidence

- **Gyromite:** The program at \$A679-\$A697 indexes a palette-choice table at \$A6FC, with 13 entries per command slot and the entries read in reverse order. Valid populated slots 1, 2, 3, 5, 6, 7, and 8 decode to the rows above. Slot 4 is deliberately skipped in the controller selection logic at \$A591-\$A5A2; its table area is not a valid optical command. Palette-selection values 3 and 4 resolve through the table at \$B35E to the all-black palette at \$B404 and all-green palette at \$B428 respectively. This ties the table bits to actual on-screen light, rather than just matching known codes by coincidence.
- **Stack-Up:** The routine at \$B1DE starts a transmission and \$B1E7-\$B237 builds successive frames. It uses the template at \$B24A: six fixed preamble bits followed by four variable bits separated by fixed green bits. The variable bits come least-significant-bit first from command values 1-6. Palette selections 2 and 3 point via \$82E8 to the black and green palettes at \$B598/\$B5AC. The ROM's final loop pass is an extra dark frame. A separate test path calls the sender with value 9 for the head light; that is not a

movement command.

- **Cross-check:** The byte patterns and action names agree with the [NESdev hardware interface](#). The original [Nintendo photosensing patent](#) describes frame-aligned light code transmission, but this report's per-game command table comes from the supplied ROMs.

These addresses refer to the CPU view of these exact 32 KiB program images (\$8000-\$FFFF). The static analysis establishes the encoded message and palette choices. It does not establish the optical fidelity of a particular emulator, TV, or camera, or every mode-specific command cadence.

Implemented decoder and camera validation

1. The OpenCV service samples a fixed central 60% region by default; `--camera-roi` changes it. It uses brightness changes, not sprite or text recognition. Automatic region finding and a second background region are not implemented.
2. The current service requests 60 fps and reports delivered fps. Synthetic tests at 60 fps with jitter accepted 667/900 valid transmissions correctly and rejected 233; zero were misclassified in that run. At 30 fps, all 900 were rejected. Actual camera/display behavior is unverified.
3. Turn samples into a frame-clocked dark/bright sequence. Search for the complete 000101 preamble, then four variable bits at the specified positions with the three required intervening green bits. Require plausible bit widths and a complete message; reject missing, merged, or ambiguous samples.
4. Accept only the action set for the identified game. A Gyromite two-level move and a Stack-Up one-level move are distinct actions. Keep the ready-light and test flashes outside the motion path. Ignore arbitrary game brightness changes that fail the full pattern.
5. A complete accepted transmission produces one controller event. Separate identical transmissions remain valid, including in Stack-Up Memory.
6. The current activity feed shows decoded and applied or blocked actions; Setup shows brightness and measured frame rate. Raw traces, bit candidates, confidence scores, and a busy queue are not exposed in the browser.

Game identification can follow the VirtualGlove-style exact launch-filename registry already described in [Game Identification](#). It selects the decoder's allowed action set and the matching virtual accessory profile; it is not evidence that a flash was decoded. A missing or unknown launch event keeps game-specific virtual actions disabled until the game is selected locally and verified.

Implemented virtual action mapping

Every accepted optical message advances **one bounded virtual primitive**. The UNO Q owns the virtual state and the browser animates it. No physical motion controller is part of this design.

Optical event	Robot operation	Display/state requirement
Left / right	Rotate one virtual station	Update station and fixture occupancy; the browser animates the resulting pose.
Gyromite up / down	Move two virtual vertical levels	Move two levels and reject an out-of-range target.
Stack-Up up / down	Move one virtual vertical level	Move one level and update held-block state.
Open / close	Open or close the illustrated grippers once	Update a piece only when the modeled grasp or release is valid.
Ready-light / test	Update optical or visual readiness only	Never move virtual pieces.

For **Gyromite**, a sequence of these primitives transfers a virtual gyro between holder, spinner, red pad, and blue pad. The UNO Q derives virtual pad state from its object model and sends that state to the paired LAN Controller 2 receiver. The flash stream alone does not identify the professor's location or prove that the game accepted the returned button. The model tracks which gyro is held and which pad is pressed.

For **Stack-Up**, the same primitives manipulate five virtual colored blocks across five numbered trays. The pose also includes six height levels; closing at a lower block can carry it and every block above it as an ordered segment. The flash stream has no block ID, target-pattern, or victory message. The UNO Q can know its own modeled stack state, but not the game's desired pattern without another validated source. Memory mode may send rapid commands; the current controller has no queue or explicit missed-command state, so this cadence needs real camera testing. In the Bingo simultaneous row-and-column case described in the manual, the game sends no command until one line changes; the camera should yield no valid movement message.

Next validation gate

Capture **all nine patterns above plus idle, test, Stack-Up Memory speeds, and the Bingo no-command case** on the intended LCD/OLED with the intended emulator and camera. Compare recovered bits against the ROM-derived rows; measure false triggers, missed bits, command spacing, and display processing delay. Until that capture succeeds, the ROM work specifies the decoder but does not prove the proposed camera will read the real screen.

CHAPTER 08 / CURRENT REFERENCE

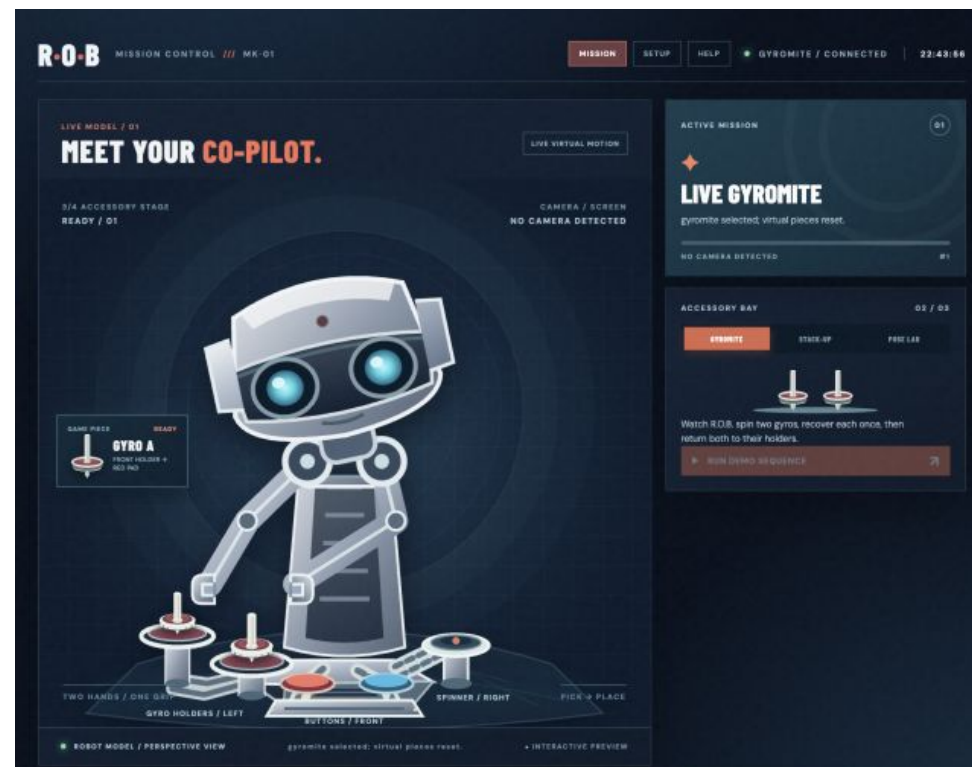
Laptop, tablet, and phone dashboard

The running Mission page puts virtual R.O.B. and the accessories first. A browser served by the UNO Q follows its controller snapshot; the same files opened directly through `file://` run the independent scripted preview. Camera framing belongs on Setup, not in the main view.

Mission layout

The large Live Model animates R.O.B.'s connected shoulders, shared opposing hands, head turn, and vertical carriage. Accessory Bay and Game Table show the same gyro or block state. Gyromite has two holders, a spinner, red/blue pads, and two gyros; Stack-Up has five trays and all five blocks. Pose Preview, Game Table, and System Vitals are grouped beneath the model. The Activity Feed distinguishes manual, optical, and link events. The selected game and RetroPie connection are separate statuses: `GYROMITE / CONNECTED` means Gyromite is selected and the browser reaches the UNO Q. The separate **RETROPIE ONLINE/OFFLINE** indicator reports receiver polling.

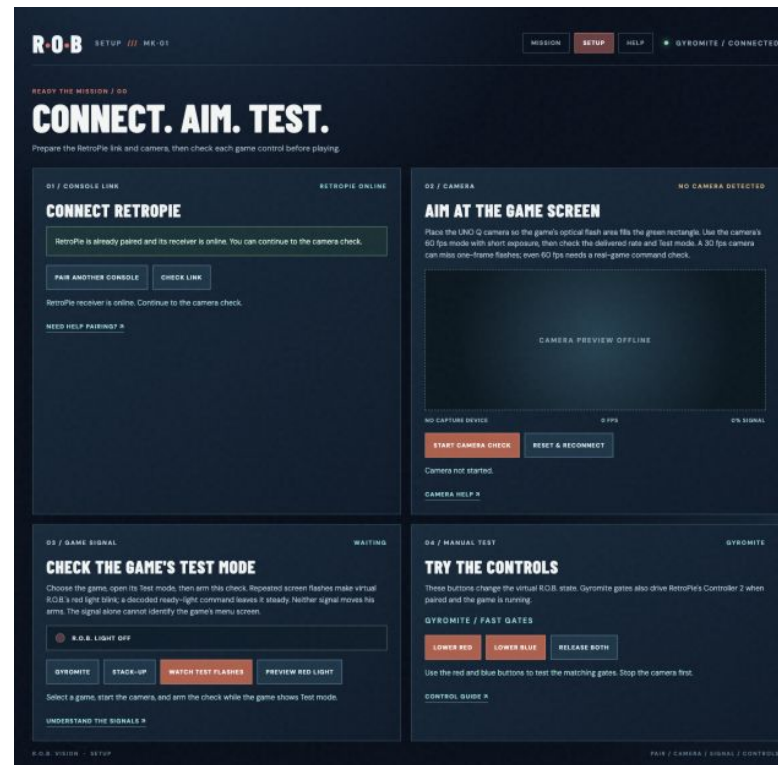
Fast Gates appears during live Gyromite when camera capture is stopped. Blue (2), red (1), and Release Both (0) act immediately; each hold expires after 60 seconds. A manual pose preview or scripted demo is not evidence of a decoded game command. Demo mode can run while RetroPie is paired and idle; a game launch stops it.





Setup layout

Setup contains RetroPie pairing and **Check Link**, camera framing and measured fps, Test-mode flash watching and red-light preview, Gyromite pad checks, and Stack-Up movement checks. The small camera image marks the sampled region. **Reset & Reconnect** reopens OpenCV capture after a disconnect; it does not reset USB hardware. Camera diagnostics remain available even though the main visual focus is R.O.B.



State and limits

dashboard/live.js polls /api/state every 500 ms and enters live mode when the controller responds. The UNO Q owns game, pose, pieces, and pad state; a browser reload simply resumes from the latest snapshot. The last 30 controller events are display history, not a guaranteed command log. Camera frames are low-rate JPEG previews for alignment, while decoding consumes timestamped frames in the controller. The browser cannot verify Hector's location, Stack-Up scoring, or a visible gate response.

The static preview demonstrates a finite Gyromite spin/return sequence, Stack-Up block transfers, and Pose Lab. Its graphics are original SVG/CSS art informed by the [historical manual notes](#). Camera-driven animation still awaits real display/camera testing.

CHAPTER 09 / CURRENT REFERENCE

R.O.B. Vision network architecture

The UNO Q owns virtual R.O.B. state. A browser on the trusted LAN renders it, and RetroPie supplies game identity and receives Gyromite pad states. The game display sends optical flashes to the UNO Q camera; no USB game-data tether is used.

```
RetroPie runcommand ■■ authenticated launch/exit HTTP ■■> UNO Q
RetroPie receiver <■■ authenticated /api/state polling ■■ UNO Q virtual pads
Browser <■■ /api/state every 500 ms ■■ UNO Q controller
Game screen ■■ optical flashes ■■> UNO Q camera (hardware validation pending)
```

The App Lab gateway serves `http://arduinoain.local` on port 80 and forwards to the controller on 8766. Existing Avahi supplies the `.local` name. Wi-Fi or an Ethernet adapter carries the same HTTP traffic. The Ethernet adapter shares the current USB hub; the supplied but uninstalled host recovery helper refuses a whole-hub reset while Ethernet is attached.

Identity and safety

Exact configured RetroPie ROM basenames select `gyromite` or `stack_up`; unknown titles clear context. The launch notifier and receiver poll use the shared token. Pairing sends that token to RetroPie after checking a short-lived code and TLS certificate fingerprint. Browser controls intentionally require no token on the trusted LAN, so neither port should be exposed publicly.

The receiver is a root uinput service that polls about every 50 ms. It releases both buttons after 750 ms without a good response, on game exit, or when no matching RetroArch process is active. It scans the running process and can resynchronize game identity after a UNO Q restart. Its authenticated poll also supplies the online indicator, which expires after three seconds.

The UNO Q's `/api/state` response is a schema-versioned snapshot with recent events. There is no push stream, durable event log, sequence protocol, or receiver acknowledgement of an on-screen gate animation. Browser refresh gets the current state. The camera preview on Setup is a low-rate framing aid and is not recorded by default. An emulator command hook remains unimplemented.

CHAPTER 10 / CURRENT REFERENCE

R.O.B. Vision UNO Q hardware setup

R.O.B., hands, gyros, spinner, pads, and Stack-Up blocks are virtual. The existing UNO Q App Lab app, matrix sketch, browser dashboard, and RetroPie link are running. The remaining physical work is a compatible camera, stable aim at the game display, and optional mounting/enclosure. No motors, mechanical robot, or accessory sensors are needed.

Use a suitable UNO Q power supply and keep its vents clear. The tested setup uses an Ethernet adapter attached through a USB hub. Wi-Fi can carry the same controller traffic. A laptop, iPad, or phone is the main R.O.B. display at <http://arduia.in.local/dashboard/>. The built-in 13x8 LED matrix supplies small status animations, not the game table.

Before buying or fixing the camera mount, identify the actual UNO Q capture device and measure delivered 30/60 fps modes, focus, exposure, and flash sampling on the intended LCD/OLED. The current code excludes codec-only `/dev/video` nodes. A camera was not attached during the engineering tests; the setup page's reconnect action retries software capture only. The optional USB recovery host helper is supplied but not installed and avoids a whole-hub reset when Ethernet shares the hub.

The RetroPie Controller 2 mapping and both Gyromite gate colors have already been verified. Commission remaining hardware in this order: attach and aim camera, observe both games' Test modes, collect repeated complete movement traces and idle false-trigger evidence, then run full interactive Gyromite and Stack-Up sessions. See [installation](#) and [verification](#).

CHAPTER 11 / CURRENT REFERENCE

R.O.B. Vision parts plan

The build is an electronic controller with a virtual R.O.B. display. Historical gyros, spinner, pads, trays, arms, and blocks are software models, not parts to buy.

Item	Current status
Arduino UNO Q	In use for App Lab controller, HTTP panel, virtual state, and LED matrix sketch.

Item	Current status
RetroPie game host	retropie.local has launch hooks and a paired virtual Controller 2; both Gyromite gate colors have responded.
Camera and adjustable mount	Needed for optical play; actual model, device node, delivered fps, exposure, and screen timing remain unverified.
UNO Q power and enclosure	Use adequate supply and ventilation; final mount depends on the chosen camera.
Wi-Fi or Ethernet adapter	Current setup uses an Ethernet adapter on a USB hub; Wi-Fi uses the same app protocol.
Laptop, tablet, or phone	Main browser display for R.O.B. and game accessories.

No servo, motor driver, home switch, physical gyro, tray sensor, spinner motor, or actuator supply belongs to this project. Do not assume a 60 fps camera will capture every one-frame game flash; validate before fixing a mount.

CHAPTER 12 / CURRENT REFERENCE

R.O.B. Vision configuration reference

Status: Current code defaults and installed test-machine behavior as of 24 September 2026. Camera and display calibration remains open.

Setting	Current value or location	Meaning
App Lab gateway	HTTP port 80	http://arduiaain.local/dashboard/ forwards to the controller.
Controller service	Port 8766	Direct /dashboard/ and /api/ access. Local CLI binds loopback by default; use --host 0.0.0.0 for a trusted LAN.
Controller token	ROB_VISION_TOKEN or UNO Q ~/.config/rob-vision/environment	Required for RetroPie launch events and authenticated receiver polls, not LAN browser controls. Keep private.

Setting	Current value or location	Meaning
Camera index	0 for manual start	OpenCV probes physical capture devices and excludes codec-only UNO Q video nodes. Actual camera node is unverified.
Camera frame request	60 fps	Delivered fps is measured and shown; a request does not guarantee that rate.
Camera region	0.2, 0.2, 0.6, 0.6	Normalized x, y, width, height; central 60% of image. CLI --camera-roi can change it.
Optical threshold/cells	Luminance 0.38; fixed 60 Hz source cells	Initial decoder values derived from ROM analysis, not calibrated to the actual display.
Gyro spin lifetime	55 seconds	Illustrative virtual lifetime.
Fast Gate hold	60 seconds maximum	Each manual press automatically releases. Fast Gates are available only when Gyromite is selected and camera capture is stopped.
Browser state polling	500 ms	/api/state snapshot; recent events capped at 30.
RetroPie receiver polling	50 ms; request timeout 250 ms	Root uinput service reads the UNO Q state.
RetroPie stale release	750 ms since last good response	Both virtual buttons release; game exit and missing RetroArch also release them.
RetroPie process scan	250 ms	Identifies active RetroArch launch and can resync UNO Q game context after restart, throttled to two seconds.
Receiver online indicator	Authenticated poll within three seconds	Otherwise Setup/Mission show RetroPie offline.
Game registry	config/games.json	Exact case-insensitive ROM basenames for Gyromite and Stack-Up, including configured ZIP, 7z, and NES names.

Local commands

```
python3 -m pip install -r requirements-camera.txt
python3 -m controller.service --host 127.0.0.1 --port 8766
python3 -m controller.service --camera-index 0 --camera-roi 0.2,0.2,0.6,0.6
python3 tools/identify_game.py start nes lr-fceumm "/roms/nes/Gyromite (World).zip" ""
python3 tools/identify_game.py end
```

The camera dependency is optional for manual controls and preview. On the UNO Q App Lab installation, OpenCV is present but no physical capture camera was attached for verification. The optional standalone `deploy/rob-vision.service` must stay disabled while App Lab owns port 8766. App Lab runs one app at a time on this device.

The repository includes UNO Q camera recovery, shutdown, early-start, and network-status host units. They are **not installed or enabled** by App Lab. Existing Avahi provides `arduiaain.local`; the R.O.B. Vision status timer is diagnostic and does not create the mDNS name. See [UNO Q host helpers](#).

Record camera model, mount, display/emulator settings, measured fps, exposure, ROI, Test-mode behavior, and decoded-command results before changing these defaults for real optical play.

CHAPTER 13 / CURRENT REFERENCE

R.O.B. Vision safety and security

R.O.B. and every accessory are virtual. There are no motors or physical gyros. The relevant risks are incorrect game input, stale network state, untrusted browser access, and camera privacy.

- Mission and Setup browser controls intentionally work without a token on the trusted LAN. Keep ports 80 and 8766 away from guest and public networks. Browser writes use same-origin JSON checks; this is not a substitute for network isolation.
- RetroPie launch notices and receiver identity require the shared controller token. Pairing uses a short-lived code and checked TLS certificate fingerprint. Do not publish the token or include it in logs.
- The optical decoder rejects incomplete and ambiguous flashes. A camera dropout does not repeat the last action. Actual display/camera performance remains unverified.
- Virtual movement validates bounds and block conservation. Invalid Stack-Up transfers do not move pieces.
- The RetroPie receiver releases both Gyromite buttons on exit, unknown game, missing process, or 750 ms without a good response. Fast Gate holds expire after 60 seconds.

- Live **Emergency Stop** stops camera capture and clears game selection. In the offline preview it only cancels the local animation. It is not a physical power switch.
- The camera preview is a low-rate framing aid on Setup. The app does not record camera video by default. Avoid aiming it at unrelated private content. User ROMs are not bundled.

The original booklet's warnings about a physical spinning gyro are historical context for Nintendo's toy, not a hazard from this virtual implementation.

CHAPTER 14 / CURRENT REFERENCE

R.O.B. Vision verification plan

Current evidence: the preview, UNO Q app, real Gyromite launch and exit identification, live red/blue pad press/release events at RetroPie's input device, a user-observed Game A gate response, and independent automated model flows for both games have been verified. See [the unattended test report](#). Optical decoding with a camera remains to be tested.

Gate	Test	Pass condition	Status
Gyromite preview	Run the unspun one-gate press, dual relay, spin-down recovery, and cleanup	Pad states stay independent; after one recovery per gyro, both gyros return upright to holders, buttons release, and COMPLETE remains stable	Browser preview observed.
Preview reset	Use Home and demo restart at different phases	Timers, queue, pieces, and buttons return to initial state	Browser implementation; regression tests pass.
Stack-Up preview	Move red alone, then blue and white together; try a blocked release	Five blocks conserved, colors stay ordered, and invalid moves leave state unchanged	Browser observed; five model tests pass.
Optical capture	Use actual game Test mode on intended LCD/OLED at measured 30 or 60 fps	Complete command decoded with timestamp evidence; idle produces no false command	Not run on hardware. Synthetic 60 fps traces missed some one-frame bits; 30 fps cannot reliably resolve them. See the timing report .

Gate	Test	Pass condition	Status
Game identity	Launch exact configured Gyromite/Stack-Up names and unknown titles	Correct context on start; idle on exit/unknown	Real Gyromite . 7z launch and exit selected then cleared UNO Q context. The receiver now recovers Gyromite context after a UNO Q restart during a live game. Both games' installed hooks passed controlled start/end tests; Stack-Up booted headlessly in RetroArch. An interactive Stack-Up launch and unknown-title launch remain pending.
Gyromite LAN output	Send red/blue virtual states and disconnect	Correct Controller 2 mapping; button releases on virtual lift, timeout, exit, and reset	Both colors pressed/released at Linux input with Player 2 forced to Gamepad. Initial isolated red moved the blue gate. After swapping A/B and restarting, isolated blue moved the blue gate and released it. The player later confirmed both red and blue controls work in Game A. Automated stale-release checks pass; sustained physical link-loss observation remains pending.
Multi-client UI	Open laptop, iPad, and phone simultaneously	Same UNO Q snapshot and ordered events; reconnection does not reset game	Not run.

CHAPTER 15 / CURRENT REFERENCE

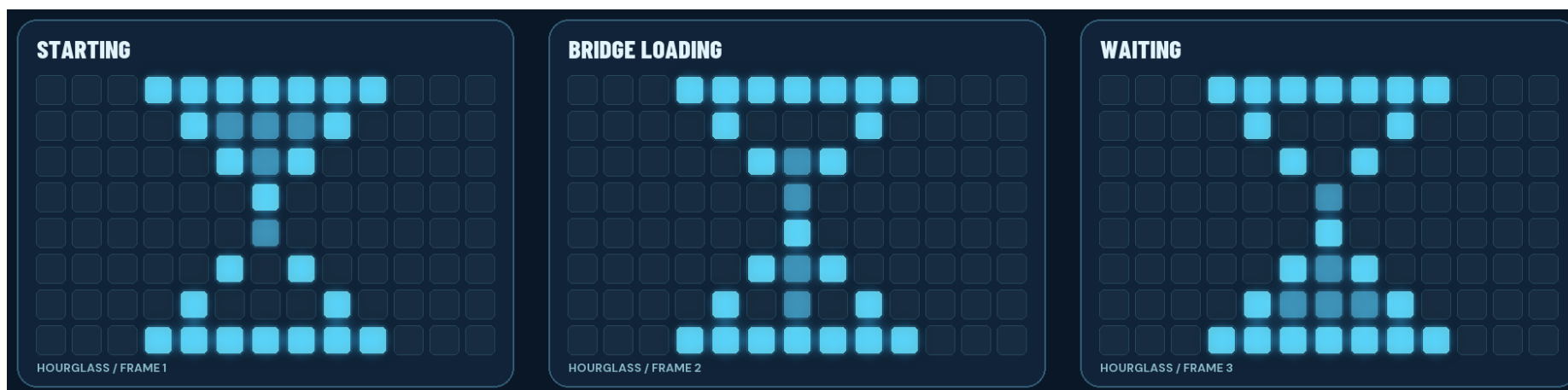
UNO Q matrix display design

Status: implemented in the R.O.B. Vision App Lab sketch and Linux bridge; physical display behavior still needs direct visual confirmation. RetroPie pairing code is generated on RetroPie and entered on the browser Setup page. The UNO Q matrix shows a pairing cue only. This guide defines the behavior for the built-in 13-column by 8-row blue LED matrix. The large browser dashboard remains the place for the full R.O.B. animation, accessories, and explanations.

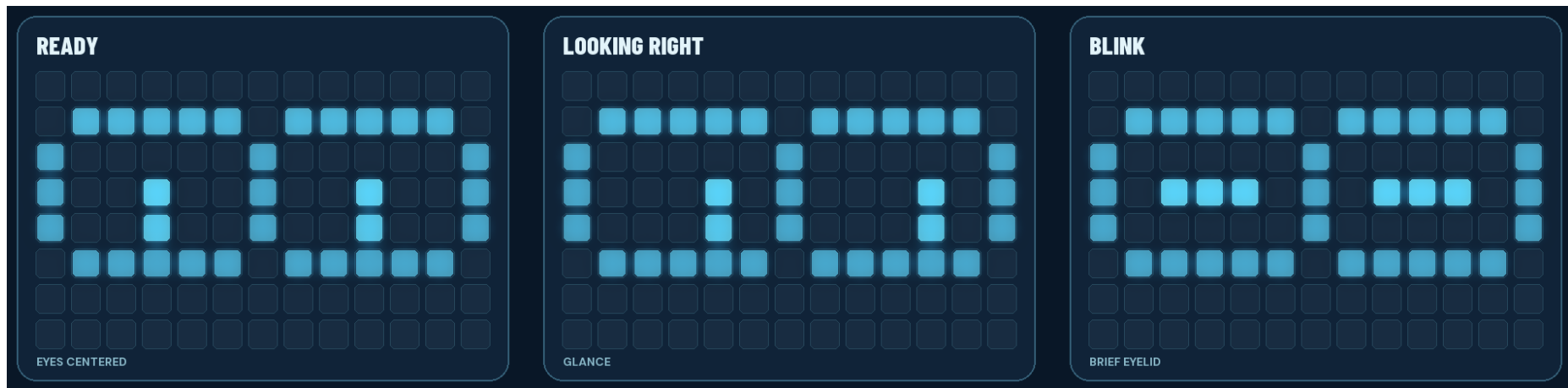
These images show the 13×8 display states defined in `sketch/sketch.ino`. Each panel is one frame of an animation.

The animations

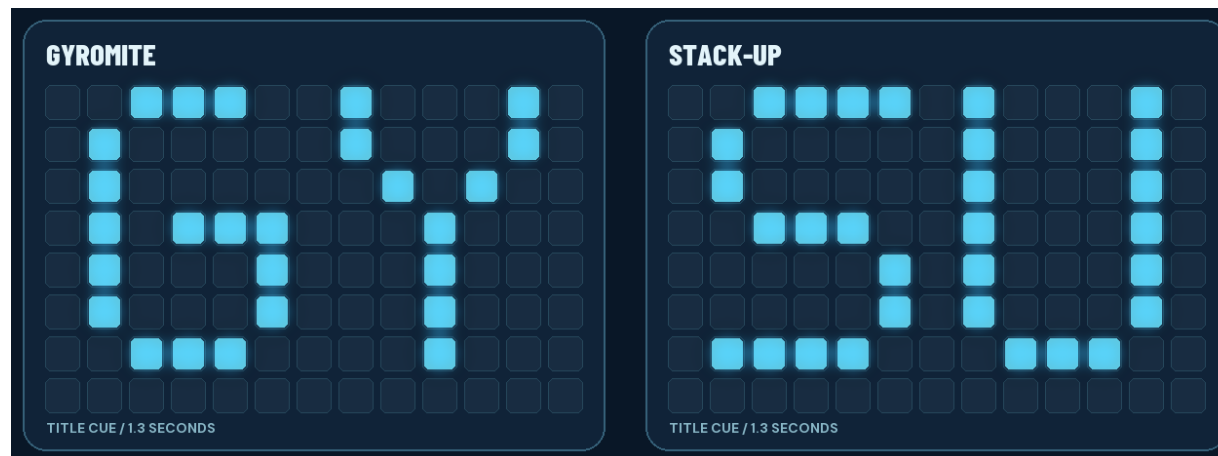
Starting and reconnecting. Three hourglass frames repeat until the controller is ready.

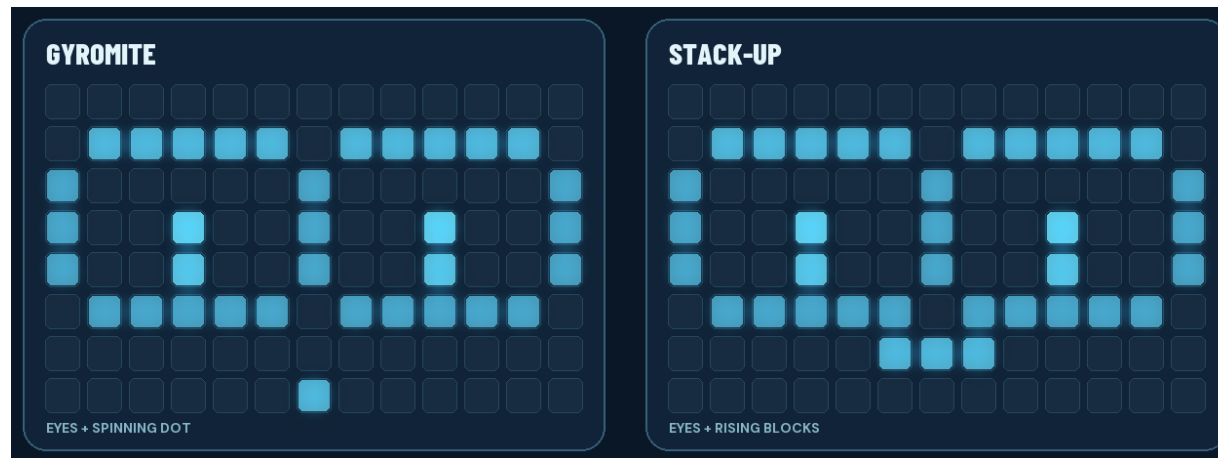


Ready, glance, blink. The eyes move slowly while R.O.B. waits for a game.

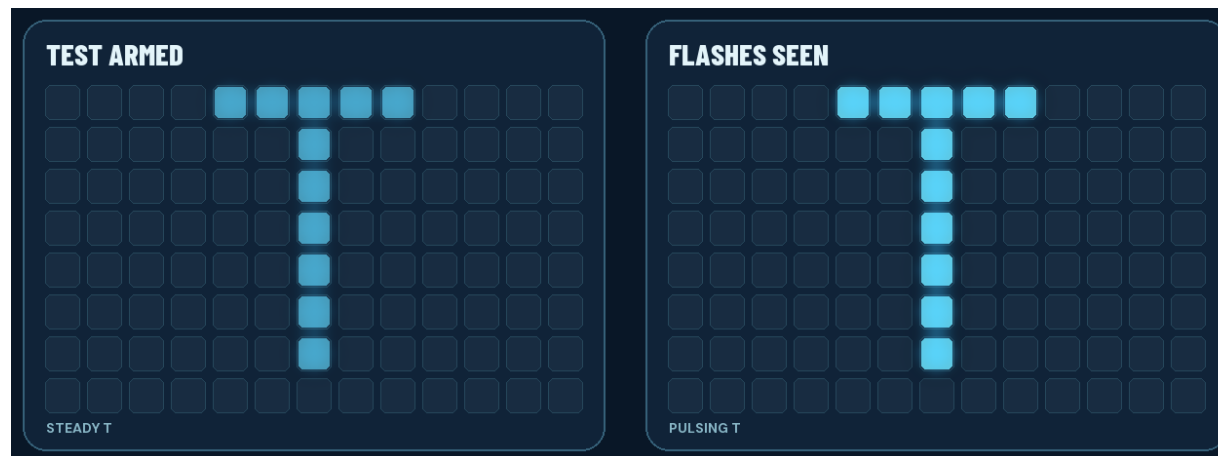


Game selected. A brief title cue identifies the game; the eyes then return with a small decorative accent. The accents do not indicate a physical gyro or block movement.



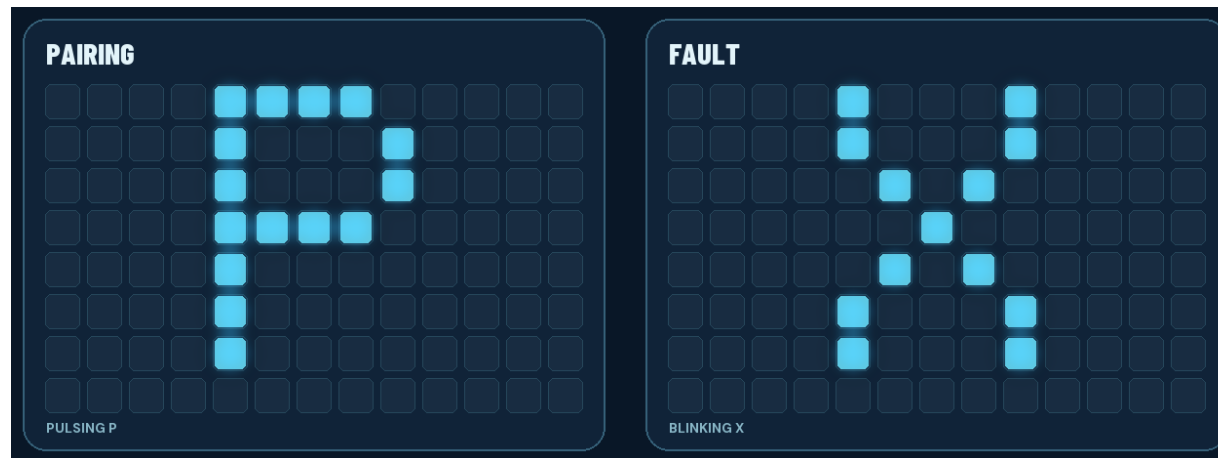


Setup and faults. The T is steady when Test is armed and gently pulses after sustained flashes are detected. P pulses during pairing; X blinks for a fault.



What people should see

State	Matrix	Meaning
Board power-on, before our sketch runs	Arduino's own boot graphics	System startup; R.O.B. Vision cannot own the matrix yet.
R.O.B. Vision sketch starts	VirtualGlove-style pulsing hourglass	The app and Linux side are starting. Begin it as early as the platform permits, before waiting for Router Bridge setup.
Linux bridge connects but the controller is still starting or reconnecting	The same hourglass, restarted or continuing smoothly	The bridge is available; the controller has not yet reported ready. A bridge connection alone must not imply game readiness.
Controller ready, no game selected	A pair of curious R.O.B. eyes: slow left/right glance and occasional blink	R.O.B. is awake and waiting for a game. This is an attract animation, not proof that a camera is attached.
Gyromite selected	Brief GY, then eyes with a small spinning-dot accent	The selected game is Gyromite. The accent is decorative; it must not claim a gyro is actually spinning.
Stack-Up selected	Brief SU, then eyes with a small rising-block accent	The selected game is Stack-Up. The accent is decorative; it must not claim a block was moved.
Valid movement command accepted	Eyes glance in the movement direction, rise/fall for vertical motion, or narrow briefly for grip	Mirror the authoritative virtual action, then return to the selected game's eye loop. Never animate from an undecoded flash alone.
Camera Test check armed	Large T	The user is checking optical alignment. The browser separately indicates whether Test flashes were actually detected.
Test flashes detected	T gently pulses	The camera has observed sustained Test-mode alternation. Avoid rapid full-matrix strobing that could reflect into the camera.
Pairing in progress	Large P with a slow pulse	Open the browser Setup page and read the pairing code and fingerprint shown on RetroPie. Do not show invented or unrelated digits on the UNO Q.
Camera or controller fault	Blinking X	Open Setup for the actual problem; the matrix alone cannot explain it.
App stopped	Matrix released/blank as the platform permits	R.O.B. Vision is no longer driving the display.



The face should be drawn for the physical 13×8 grid, with two distinct eyes and enough dark space to make a blink or sideways glance readable. The blue LEDs are monochrome: eye shapes and brightness, not color, carry the expression. Limit idle motion to an occasional glance or blink so it feels alive without competing with the game or affecting the camera. Show GY/SU long enough to identify the title, then let the eyes take over. A brief repeat of the title mark after a long idle period is acceptable.

State ownership and priority

The sketch owns all framebuffer writes and animates the startup hourglass without waiting for Linux. Linux sends compact requested states and accepted action hints through Router Bridge, using the UNO Q controller's authoritative game, camera, test, and fault state. It sends a heartbeat at least once per second. If Linux disappears for about 3.5 seconds, the display returns to the hourglass rather than freezing a misleading game face. The sketch cannot replace Arduino's protected boot logo before App Lab releases the microcontroller.

Display priority: **fault** → **pairing** → **Test check** → **startup/reconnect** → **selected game** → **idle eyes**. A movement hint is temporary within the selected game and never overrides fault, pairing, or Test. A camera being absent should be visible on Setup; the idle eyes alone must not claim optical readiness. Pairing has a bounded lifetime and returns to the prior state when complete or expired.

Pairing distinction

VirtualGlove has a separate physical pairing display for its own device approval PIN. R.O.B. Vision pairs **RetroPie to the UNO Q**: RetroPie produces a six-digit code and certificate fingerprint, while the UNO Q browser Setup page accepts them. The matrix P is a status cue when **Pair another console** is opened or pairing is submitted; it expires or clears afterward. Showing the code on the UNO Q would require a deliberate change to the pairing protocol.

Physical acceptance checks still needed

1. Cold boot shows the protected system graphics, then the hourglass as soon as the R.O.B. Vision sketch can run; the hourglass remains through Router Bridge initialization.
2. When the Linux controller is ready, the hourglass changes to the idle eyes. Stopping/restarting Linux restores the hourglass and then the correct current game display.
3. Real Gyromite and Stack-Up launch/exit notifications select and clear their distinct marks and eye loops.
4. Test T, pairing P, and fault X follow the priority and timeout rules. No matrix state claims a camera signal or game action that the controller has not observed.
5. Verify brightness and animation beside the intended camera/display setup to ensure matrix reflections do not trigger optical recognition.